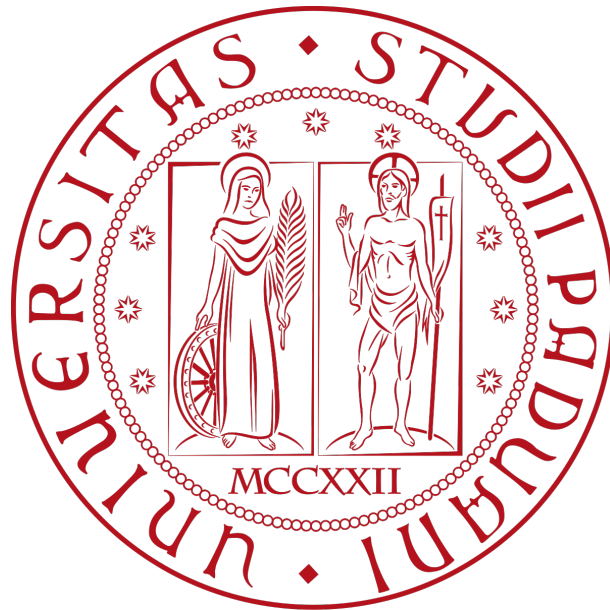


University of Pauda



AIoT Basics using Containerization

Exercises

Student: *Lorenzo Albertin*
Student ID: *220160*

ACCADEMIC YEAR 2025-2026

Indice

1	Exercise 1 - EdgeImpulse	1
2	Exercise 2 - TIG Dashoboard	2
3	Exercise 3 - MQTT Brokers	3
4	Exercise 4 and 5: Telegram Bot	4

1 Exercise 1 - EdgeImpulse

To complete the first exercise, I decided to classify the same items used in the provided demo. In particular, I took 20 pictures of a coin, 20 pictures of a pen and 20 pictures of the background.

After the training with an 80/20 split, I obtained the following results:



Figura 1: Training results

I also tested the model on the validation set, achieving an accuracy of 100% as shown in the following screenshot:

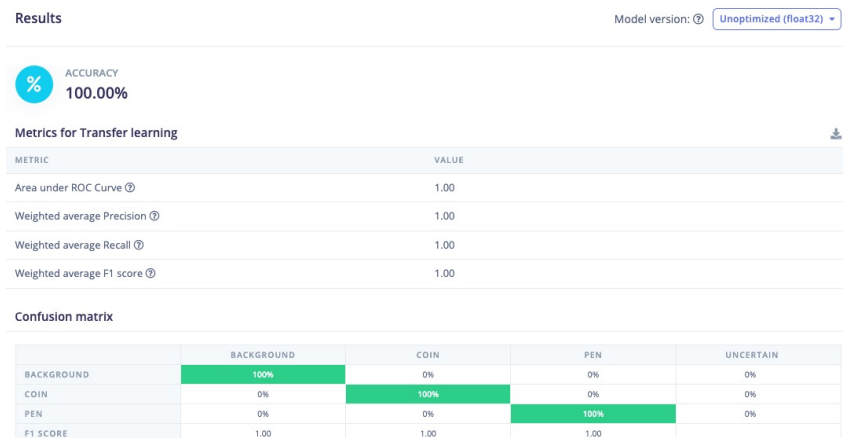


Figura 2: Validation results

To test the model on your phone, you can scan the following QR code:



Figura 3: EdgeImpulse model QR code

2 Exercise 2 - TIG Dashoboard

I built a dashboard to monitor the data received from the MQTT server at `"tcp://eu1.cloud.thethings.network:1883"`.

In particular, I used Telegraf to collect the data coming from the MQTT Server and InfluxDB to store it. Then, using Grafana, I created a dashboard to visualize the data. All components were running inside Docker containers.

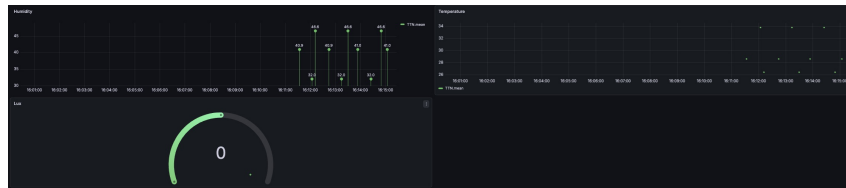


Figura 4: Grafana dashboard

3 Exercise 3 - MQTT Brokers

For this exercise, I wrote a docker-compose file to run 2 MQTT brokers simultaneously in 2 different containers. In particular:

- Each broker has its own container name;
- broker_a is accessible at 'localhost:1883';
- broker_a is accessible at 'localhost:1884'.

```
services:
  broker_a:
    image: eclipse-mosquitto:latest
    container_name: broker_a
    ports:
      - "1883:1883"
    volumes:
      - ./mosquitto:/mosquitto
    networks:
      - sub-net

  broker_b:
    image: eclipse-mosquitto:latest
    container_name: broker_b
    ports:
      - "1884:1883"
    volumes:
      - ./mosquitto:/mosquitto
    networks:
      - sub-net

networks:
  sub-net:
    driver: bridge
```

You can find the docker-compose file showed above, visiting the following link:

<https://github.com/lalb03/Assignments-AIoT-2025-26/tree/9867300dca86d1b2c51edd>

MQTT

4 Exercise 4 and 5: Telegram Bot

Following the guide, I developed a Telegram bot that uses the *Open-Meteo API* to fetch the current weather in Padua.

In particular, every time a user sends the command `"/getdata"`, the Telegram bot makes an HTTP call to the *Open-Meteo API*. Once it receives the response, it sends the data back to the user and stores the measurement in a local database.

All available commands are the following:

- `"/start"`: starts the conversation and shows the list of available commands;
- `"/getdata"`: retrieves the current temperature in Padua;
- `"/history"`: returns the last 10 measurements;
- `"/average"`: returns the average temperature over all measurements stored in the database;
- `"/max"`: returns the maximum temperature stored in the database;
- `"/min"`: returns the minimum temperature stored in the database.

The code of the bot is available at the following link: [https://github.com/](https://github.com/lalb03/Assignments-AIoT-2025-26/blob/b01b02599151ea6dd1423e50dac6728c923c3610/)

[lalb03/Assignments-AIoT-2025-26/blob/b01b02599151ea6dd1423e50dac6728c923c3610/](https://github.com/lalb03/Assignments-AIoT-2025-26/blob/b01b02599151ea6dd1423e50dac6728c923c3610/)

`Telegram_bot/tel_ex-2.py`

In the following page, you can find a screenshot of a conversation with the bot.

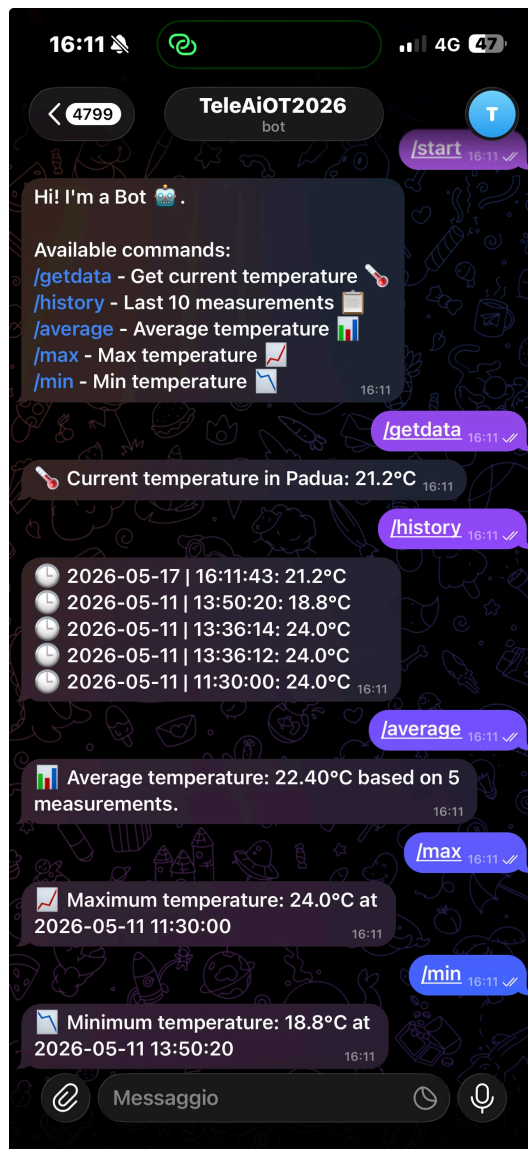


Figura 5: Screenshot of a conversation